Apache POI i Commons CSV

Kompendium Wiedzy i Scenariusz Prezentacji

Część 1. Podstawa Teoretyczna

Ten przewodnik pomoże Ci zrozumieć, czym są biblioteki Apache POI oraz Apache Commons CSV, do czego służą w ekosystemie Javy i jakie mają typowe zastosowania. Są to kluczowe narzędzia podczas mapowania danych wejściowych z plików na obiekty Javy (DTO / rekordy) oraz przy generowaniu eksportów.

1. Excel vs CSV - Co wybrać i kiedy?

Zarówno CSV, jak i Excel (.xlsx) służą do przechowywania danych tabelarycznych, jednak ich natura narzuca zupełnie inne scenariusze użycia.

Cechy	CSV (Comma-Separated Values)	Excel (.xlsx)
Struktura i Typy	Płaski tekst, brak typów (wszystko to String), brak zakładek/arkuszy.	Plik binarny (archwium XML), obsługuje arkusze, style, formuły i natywne typy danych.
Wydajność	Lekki, idealny do strumieniowania i wielkich plików (miliony wierszy).	Ciężki dla pamięci, wczytanie całego pliku do RAM może spowodować `OutOfMemoryError`.
Reguła kciuka	Integracje systemów i ETL (Extract, Transform, Load). Transfer danych do baz.	Raporty dla użytkowników biznesowych. Wymagane formatowanie i czytelność.

2. Zależności i Mapowanie na DTO

Obie biblioteki nie są wbudowane w standardowe JDK, należy je dodać poprzez Maven (grupy `org.apache.poi` oraz `org.apache.commons`). Celem korzystania z nich najczęściej nie jest manipulacja komórkami sama w sobie, ale transformacja danych ze świata zewnętrznego na obiekty Javy. We współczesnej Javie do tego celu idealnie nadają się Rekordy (Records) lub tradycyjne DTO.

```
public record Person(String name, LocalDate birthDate, BigDecimal salary) {}
```

Użycie `BigDecimal` zamiast `double` dla wartości pieniężnych to kluczowa dobra praktyka - chroni przed utratą precyzji w obliczeniach zmiennoprzecinkowych.

3. Apache Commons CSV

CSV to z pozoru prosty format, co często kusi programistów do ręcznego parsowania wierszy (np. `String.split(";")`). Jest to poważny błąd. Ręczne rozdzielanie sypie się natychmiast, gdy wewnątrz samego pola tekstowego pojawi się znak separatora, cudzysłów lub znak nowej linii. Od tego są biblioteki takie jak Commons CSV.

Odczyt plików i Dialekty: Biblioteka opiera się na definicji Dialektu (`CSVFormat`). Domyślnie formatuje się go przez Buildera, gdzie możemy wskazać separator, nagłówki czy zasady cytowania.

```
CSVFormat format = CSVFormat.DEFAULT.builder() .setDelimiter(';') // W polskim Excelu separatorem jest często średnik! .setHeader() // Deklaracja, że plik posiada nagłówki .withFirstRecordAsHeader() // Użycie pierwszej linii jako mapy nagłówków .withIgnoreEmptyLines(true) .build(); try (Reader reader = new FileReader("data.csv", StandardCharsets.UTF_8); CSVParser parser = format.parse(reader)) { // Walidacja obecności kolumny (ochrona przed wybuchem kodu) if (!parser.getHeaderMap().containsKey("salary")) { throw new IllegalArgumentException("Brak wymaganej kolumny 'salary'"); } for (CSVRecord record : parser) { String name = record.get("name"); // parsowanie liczb i dat... } }
```

Typowe pułapki w CSV:

- **Zły delimiter:** Polski Excel często zapisuje CSV ze średnikiem (;), a nie przecinkiem. Trzeba o to dbać przy użyciu `setDelimiter()`.
- **Kodowanie i znacznik BOM:** Jeśli używasz UTF-8, Windowsowy Excel często wymaga na początku pliku znacznika BOM, aby poprawnie wyświetlać polskie znaki diakrytyczne.
- **Różne nazwy nagłówków:** Różnica wielkości liter powoduje błędy.
- **Mini-test wejścia (Sanity check):** Przy odczycie wielkich plików warto wczytać pierwsze 10 rekordów by zweryfikować poprawność danych, unikając przetwarzania 2 GB "śmieci".

4. Apache POI - Zrozumieć strukturę Excela

Apache POI operuje na obiektowym modelu dokumentu. Dla plików `.xlsx` używa się implementacji rozpoczynających się od XSSF. Hierarchia klas jest logiczna i odzwierciedla strukturę arkusza kalkulacyjnego: **Workbook** (Cały plik) → **Sheet** (Pojedyncza zakładka) → **Row** (Wiersz) → **Cell** (Komórka).

Odczyt danych i Problem z Typami:

```
Workbook workbook = new XSSFWorkbook(new FileInputStream("data.xlsx")); Sheet sheet = workbook.getSheetAt(0); DataFormatter formatter = new DataFormatter(); // "Cywilny" odczyt for (Row row : sheet) { if (row.getRowNum() == 0) continue; // Pomijamy nagłówek Cell nameCell = row.getCell(0); // Odczyt komórki dokładnie tak, jak widzi ją człowiek w Excelu: String nameText = formatter.formatCellValue(nameCell); Cell dateCell = row.getCell(1); // Jeśli data to "prawdziwa" data w Excelu, pobieramy ją poprawnie: LocalDateTime date = dateCell.getLocalDateTimeCellValue(); }
```

Daty, Formuły i Kasa: Excel pod spodem przechowuje daty jako liczby zmiennoprzecinkowe (ilość dni od 1900 roku). Należy zawsze używać `getLocalDateTimeCellValue()`. Formuły (np. =A1+B1) nie trzymają wyniku zaktualizowanego - do ich bezpiecznego odczytania należy wywołać `FormulaEvaluator`.

5. Zapis w POI i praca z gigantycznymi plikami

Jeśli spróbujesz zapisać 2 miliony rekordów do Excela używając standardowego `XSSFWorkbook`, aplikacja pochłonie cały RAM. Apache POI oferuje tryb Streaming (SXSSF) przeznaczony specjalnie do wielkich plików.

```
// "100" to rozmiar okna - w pamięci trzyma tylko 100 wierszy naraz try (SXSSFWorkbook sxssfWorkbook = new SXSSFWorkbook(100)) { Sheet sheet = sxssfWorkbook.createSheet("Raport"); // ... generowanie milionów wierszy try (FileOutputStream out = new FileOutputStream("wielki_raport.xlsx")) { sxssfWorkbook.write(out); } // Ważne! Czyszczenie plików tymczasowych z dysku: sxssfWorkbook.dispose(); }
```

6. Podsumowanie Dobrych Praktyk

- **CSV - Zawsze jawne parametry:** Trzymaj konfigurację w jednym miejscu (ustalony delimiter, obsługa BOM).
- **CSV - Streamuj wejście:** Zawsze przetwarzaj iteracyjnie. Nie ładuj zawartości parsera do `List<Record>` przy ogromnych plikach.
- **POI - Optymalizuj style:** Liczba formatowań w `.xlsx` jest limitowana. Twórz obiekty `CellStyle` na poziomie skoroszytu, a nie dla każdej komórki z osobna!
- **Zamknięcie zasobów:** Wszystko musi być otwierane w blokach `try-with-resources`.

Część 2. Propozycja Prezentacji (Dla 2 Prelegentów)

Poniżej znajduje się dostosowany scenariusz prezentacji. Został zoptymalizowany dla dokładnie **dwóch osób**. Wprowadzono też obowiązkowe **Live Demo**, łączące obie technologie w praktyce.

Slajd 1: Wstęp i cel prezentacji

Osoba 1

- Cel: Wygodne mapowanie plików `.xlsx` oraz `.csv` na obiekty Javy.
- Problem: Parsowanie tekstu to pułapki; pliki Excela to tak naprawdę paczki XML.
- Rozwiązanie: Stabilne biblioteki Open Source (Maven: `org.apache.poi`, `commons-csv`).

Notatka: Przywitaj grupę. Wyjaśnij, że dzisiaj skupimy się na bezpiecznym czytaniu i pisaniu plików w produkcyjnym kodzie.

Slajd 2: Kiedy CSV, a kiedy Excel?

Osoba 1

- **CSV:** Lekki, szybki, łatwy do strumieniowania. Idealny do procesów ETL i integracji.
- **Excel (.xlsx):** Obsługuje typy, style i formuły. Idealny do raportów biznesowych.

Notatka: Złota reguła: proces-do-procesu = CSV; proces-do-człowieka (księgowość) = Excel.

Slajd 3: Apache Commons CSV - odczyt

Osoba 1

- Definiowanie formatu: `CSVFormat.DEFAULT.builder()`
- Ważne modyfikatory: `.setDelimiter(';')`, `.setHeader()`
- Iteracyjne czytanie zamiast pakowania wszystkiego do Listy.

Notatka: Pokaż krótki urywek kodu Buildera. Podkreśl, dlaczego nie używamy tu Listy przy plikach rzędu 500MB.

Slajd 4: Pułapki formatu CSV

Osoba 1

- Ręczne parsowanie (`split(",")`) niszczy dane z wewnętrznymi przecinkami.
- Zły separator (polski Excel domyślnie używa średnika!).
- Brak BOM (krzaczki zamiast polskich znaków w Excelu na Windowsie).

Notatka: Opowiedz anegdotę o wybuchającym kodzie rozdzielającym stringi, gdy klient wpisał swój adres z przecinkiem w środku pola.

Slajd 5: Apache POI - Architektura (Odczyt)

Osoba 2

- Logiczna hierarchia: **Workbook** → **Sheet** → **Row** → **Cell**
- `DataFormatter` to klasa ratująca życie (wyciąga widok wizualny komórki do Stringa).

Notatka: Osoba 2 przejmuje pałeczkę. Wyjaśnij, że POI traktuje Excela obiektowo, a komórki bywają "nullowe", jeśli są puste.

Slajd 6: Excelowe Daty i Formuły

Osoba 2

- Daty to liczby zmiennoprzecinkowe! Używaj `getLocalDateTimeCellValue()`.
- Formuły wymagają `FormulaEvaluator` (wartość nie odświeża się magicznie w Javie).

Notatka: Mocno zaakcentuj problem "dziwnych liczb" zamiast dat, to najczęstszy błąd początkujących.

Slajd 7: Duże pliki (SXSSF) i Precyzja

Osoba 2

- `XSSFWorkbook` wywala pamięć przy milionie wierszy.
- Używamy klas z literą **S** (Streaming): `SXSSFWorkbook`.
- Finanse (pieniądze) = `BigDecimal` w kodzie, formatowanie księgowe w Excelu (żadnych `double`!).

Notatka: Przypomnij o konieczności użycia dispose() na koniec, żeby skasować pliki tymczasowe wygenerowane przez SXSSF.

Slajd 8: Pułapki stylów w Excelu

Osoba 2

- Generowanie raportu z nałożonymi stylami bywa trudne.
- **Błąd krytyczny:** Tworzenie nowego `CellStyle` dla każdej komórki w pętli.
- Skutek: Excel wariuje przy ~64 000 stylów. Rozwiązanie: Jeden obiekt stylu na globalnym poziomie Workbook.

Notatka: Krótki wtręt dla zaawansowanych – pokazuje głęboką wiedzę prelegentów.

Slajd 9: LIVE DEMO - Połączenie sił (Konwerter CSV → Excel)

Osoba 1 & Osoba 2

Odczytujemy listę pracowników z CSV i zapisujemy elegancki raport .xlsx.

```
// OSOBA 1: Odczyt z Commons CSV Reader reader = new FileReader("pracownicy.csv");
Iterable<CSVRecord> records =
CSVFormat.DEFAULT.builder().setHeader().build().parse(reader); // OSOBA 2: Zapis do
Apache POI Workbook workbook = new XSSFWorkbook(); Sheet sheet =
workbook.createSheet("Raport Pracowników"); int rowNum = 0; for (CSVRecord record :
records) { Row row = sheet.createRow(rowNum++);
row.createCell(0).setCellValue(record.get("Imie"));
row.createCell(1).setCellValue(record.get("Stanowisko")); } try (FileOutputStream out =
new FileOutputStream("raport_koncowy.xlsx")) { workbook.write(out); }
```

Notatka: Osoba 1 krótko omawia 2 linijki czytania CSV. Następnie Osoba 2 tłumaczy pętlę i zapis za pomocą POI. Kod jest super prosty, ale pokazuje istotę "przepisywania" z jednego narzędzia do drugiego.

Slajd 10: Podsumowanie (Dobre Praktyki)

Osoba 1

- Bloki `try-with-resources` to obowiązek.
- `DataFormatter` i `BigDecimal` chronią integralność danych.
- Dla wielkich plików: "Sanity check" kilku pierwszych linijek i iteracyjne przetwarzanie.

Notatka: Zostawić słuchaczy z wnioskiem: narzędzia są potężne, ale zapomnienie o limitach RAMu skończy się źle.

Slajd 11: Q&A

Osoba 2

- Dziękujemy za uwagę!
- Materiały i kod źródłowy z demem: [link_do_repozytorium]
- Czy macie jakieś pytania?

Notatka: Ostatni slajd wyświetlany podczas sesji pytań. Link zostaje na ekranie.